

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỆ THỐNG NHÚNG (TN) (CO3054)

BÀI TẬP LỚN

TRÒ CHƠI BRICK BREAKER

GVHD: Trần Nguyễn Minh Duy

Nhóm SV thực hiện: Trần Tấn Hưng – 2211386
Lê Quang Trung – 2213730
Nguyễn Duy Khiêm – 2211571
Nguyễn Minh Hưng – 2211367
Nguyễn Văn Huỳnh – 2211315
Phan Lê Hậu – 2210969

Thành phố Hồ Chí Minh, tháng 11 năm 2025

Mục lục

Danh sách hình vẽ	3
DANH SÁCH THÀNH VIÊN	4
1 Giới thiệu	5
1.1 Lý do chọn đề tài	5
1.2 Bối cảnh ứng dụng thực tế	5
1.3 Mục tiêu của hệ thống	6
1.4 Phạm vi và giới hạn	6
2 Phân tích yêu cầu hệ thống	7
2.1 Yêu cầu chức năng	7
2.1.1 Chức năng thu thập tín hiệu điều khiển (Input)	7
2.1.2 Chức năng hiển thị và đồ họa (Visual Output)	7
2.1.3 Chức năng xử lý logic và âm thanh (Processing & Audio)	7
2.2 Yêu cầu phi chức năng	7
2.3 Môi trường hoạt động	8
3 Thiết kế hệ thống	9
3.1 Sơ đồ khối tổng thể của hệ thống	9
3.2 Thiết kế luồng thực thi của hệ thống dựa vào Máy trạng thái (FSM)	10
3.3 Thiết kế Flowchart cho xử lý logic game	12
3.3.1 Flowchart Main Loop (<code>step_world</code>)	12
3.4 Flowchart Wall Collision (<code>resolve_ball_wall</code>)	13
3.5 Flowchart Paddle Collision (<code>resolve_ball_paddle</code>)	14
3.6 Flowchart Brick Collision (<code>resolve_ball_brick</code>)	15
3.7 Thiết kế Testcase cho hệ thống	16
3.8 Thiết kế giao diện cho hệ thống	19
3.8.1 Tổng quan về Bố cục và Các Thành phần Chính	19
3.8.2 Mô tả các Màn hình Giao diện	19
4 Triển khai	23
4.1 Mô tả chi tiết phần cứng sử dụng	23
4.2 Công cụ phát triển	23
4.3 Mã nguồn hiện thực chương trình	24
5 Kết quả và đánh giá	25
5.1 Kết quả Demo	25
5.2 Đánh giá hiệu năng	25
5.3 So sánh với mục tiêu ban đầu	26



6	Kết luận và hướng phát triển	27
6.1	Tổng kết điều đạt được	27
6.2	Các giới hạn của hệ thống	27
6.3	Đề xuất cải tiến	27
7	Tài liệu tham khảo	29

Danh sách hình vẽ

1	Sơ đồ khối tổng thể của hệ thống	9
2	Sơ đồ máy trạng thái của hệ thống	10
3	Flowchart xử lý logic cho mỗi khung hình	12
4	Flowchart xử lý va chạm với biên màn hình	13
5	Flowchart xử lý va chạm giữa bóng và thanh đỡ (<code>resolve_ball_paddle</code>) sử dụng kỹ thuật AABB	14
6	Flowchart xử lý va chạm và phản xạ với gạch (<code>resolve_ball_brick</code>)	15
7	Quy trình kiểm thử hệ thống	16
8	Màn hình giới thiệu	19
9	Hình ảnh Hướng Dẫn chơi game	20
10	Hình ảnh chơi game	21
11	Hình Ảnh Pause Game	21
12	Hình Ảnh Game Over	22

DANH SÁCH THÀNH VIÊN

No.	Họ và tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
1	Trần Tấn Hưng	2211386	- Rút môn	0%
2	Lê Quang Trung	2213730	- Thiết kế kiến trúc hệ thống - Viết báo cáo latex	100%
3	Nguyễn Duy Khiêm	2211571	- Thiết kế và hiện thực logic của trò chơi - Viết báo cáo latex	100%
4	Nguyễn Minh Hưng	2211367	- Mô tả hệ thống và mục tiêu - Thiết kế UI	100%
5	Nguyễn Văn Huynh	2211315	- Thiết kế kiến trúc hệ thống - Viết báo cáo latex	100%
6	Phan Lê Hậu	2210969	- Thiết kế testing - Xử lý biến trở điều khiển Paddle, tích hợp Buzzer	100%

Bảng 1: Danh sách thành viên

1 Giới thiệu

1.1 Lý do chọn đề tài

Trong kỷ nguyên công nghệ 4.0, hệ thống nhúng (Embedded Systems) đóng vai trò cốt lõi trong hầu hết các thiết bị điện tử, từ dân dụng đến công nghiệp. Việc làm chủ các dòng vi điều khiển 32-bit hiệu năng cao như ARM Cortex-M (cụ thể là STM32) là yêu cầu cấp thiết đối với sinh viên ngành kỹ thuật máy tính và điện tử.

Để hiện thực hóa các kiến thức lý thuyết về lập trình nhúng, nhóm em thực hiện chọn đề tài "Xây dựng trò chơi Brick Breaker trên Kit phát triển BKIT-Arm4". Lý do lựa chọn đề tài này bao gồm:

- Tính tổng hợp cao: Dự án yêu cầu kết hợp chặt chẽ giữa lập trình phần cứng (giao tiếp ngoại vi ADC, PWM, SPI/FSMC, I2C) và tư duy thuật toán (logic game, xử lý va chạm, quản lý bộ nhớ).
- Tính trực quan và tương tác: Khác với các bài tập điều khiển LED đơn thuần, việc xây dựng một trò chơi yêu cầu phản hồi thời gian thực (Real-time response) giữa thao tác người dùng và hiển thị đồ họa, tạo động lực học tập lớn.
- Nền tảng cho các ứng dụng HMI: Dự án là bước đệm vững chắc để tiếp cận việc xây dựng các giao diện người - máy (Human-Machine Interface) phức tạp hơn trong tương lai.

1.2 Bối cảnh ứng dụng thực tế

Mặc dù "Brick Breaker" là một trò chơi cổ điển, nhưng kiến trúc và kỹ thuật xây dựng hệ thống này mang tính ứng dụng thực tế cao trong công nghiệp và đời sống:

- Thiết bị giải trí cầm tay (Handheld Consoles): Mô phỏng nguyên lý hoạt động của các máy chơi game cầm tay, nơi vi điều khiển xử lý đồng thời đồ họa, âm thanh và tín hiệu điều khiển từ người dùng.
- Giao diện điều khiển công nghiệp: Cơ chế sử dụng biến trở để điều chỉnh thông số hiển thị trên màn hình LCD (tương tự việc điều khiển thanh trượt trong game) được ứng dụng rộng rãi trong các bảng điều khiển máy móc, thiết bị y tế, hoặc bộ điều chỉnh âm thanh/ánh sáng.

1.3 Mục tiêu của hệ thống

Mục tiêu chính của đề tài là thiết kế và chế tạo thành công hệ thống trò chơi Brick Breaker hoạt động ổn định trên Kit BKIT-Arm4 với các yêu cầu cụ thể sau:

- Mục tiêu về chức năng:
 - Xây dựng hoàn chỉnh gameplay với các quy tắc: điều khiển paddle đỡ bóng, phá gạch tính điểm, quản lý mạng chơi (lives) và chuyển cấp độ (level).
 - Tích hợp tính năng mở rộng (Power-ups).
- Mục tiêu về kỹ thuật:
 - Điều khiển hiển thị: Sử dụng thành thạo giao tiếp (FSMC/SPI) để điều khiển màn hình TFT LCD (ILI9341), hiển thị đồ họa mượt mà, không bị giật/nhảy.
 - Xử lý tín hiệu vào (Input): Đọc chính xác giá trị Analog từ biến trở qua bộ chuyển đổi ADC để điều khiển vị trí paddle; xử lý tín hiệu số từ nút nhấn (GPIO) để quản lý trạng thái game.
 - Xử lý tín hiệu ra (Output): Tạo hiệu ứng âm thanh tương ứng với các sự kiện trong game bằng phương pháp điều chế độ rộng xung (PWM) điều khiển Buzzer.

1.4 Phạm vi và giới hạn

Dựa trên yêu cầu thiết kế và phần cứng hiện có, đề tài được giới hạn trong phạm vi sau:

- Phạm vi phần cứng: Hệ thống được triển khai duy nhất trên Board thí nghiệm BKIT-Arm4 sử dụng vi điều khiển STM32. Các ngoại vi bao gồm: LCD ILI9341, Biến trở, Nút nhấn tích hợp, Buzzer.
- Phạm vi tính năng:
 - Trò chơi là Single-player (một người chơi).
 - Đồ họa ở mức 2D cơ bản trên độ phân giải 240x320 pixel.
 - Âm thanh dạng đơn âm (beep tones), không hỗ trợ phát nhạc chất lượng cao (MP3/WAV).
- Môi trường thử nghiệm: Hệ thống được thử nghiệm trong điều kiện phòng thí nghiệm, nguồn điện ổn định từ board mạch, chưa tính đến các yếu tố nhiễu công nghiệp phức tạp hoặc thiết kế vỏ hộp sản phẩm thương mại.

2 Phân tích yêu cầu hệ thống

2.1 Yêu cầu chức năng

Hệ thống cần đảm bảo thực hiện chính xác các nhóm chức năng sau đây để trò chơi vận hành trơn tru:

2.1.1 Chức năng thu thập tín hiệu điều khiển (Input)

- **Đọc tín hiệu Analog (Paddle Control):** Hệ thống phải đọc liên tục giá trị điện áp từ biến trở thông qua bộ chuyển đổi ADC (cụ thể là chân PC1). Giá trị đọc được (0 – 4095) phải được ánh xạ (mapping) chính xác sang tọa độ ngang (x) của màn hình để điều khiển thanh trượt (paddle).
- **Đọc trạng thái nút nhấn (State Control):** Hệ thống phải nhận diện được tín hiệu số (Digital Input) từ các nút nhấn tích hợp trên board (GPIO) để thực hiện các lệnh. Cần tích hợp giải thuật khử rung (debouncing) để đảm bảo tín hiệu nút nhấn ổn định.

2.1.2 Chức năng hiển thị và đồ họa (Visual Output)

- **Vẽ đối tượng game:** Hiển thị đầy đủ các thành phần của game trên màn hình LCD (ILI9341) thông qua giao tiếp FSMC hoặc SPI. Các đối tượng bao gồm: Thanh trượt (Paddle), Quả bóng (Ball), và Ma trận gạch (Bricks).
- **Cập nhật giao diện (UI):** Hiển thị thông tin trạng thái theo thời gian thực: Điểm số (Score), Số mạng còn lại (Lives), Level hiện tại.
- **Hiệu ứng hình ảnh:** Hiển thị các hiệu ứng trực quan khi bóng va chạm, gạch vỡ, hoặc khi nhận được vật phẩm hỗ trợ (Power-ups).

2.1.3 Chức năng xử lý logic và âm thanh (Processing & Audio)

- **Xử lý va chạm (Collision Detection):** Vi điều khiển phải tính toán liên tục vị trí của bóng để phát hiện va chạm với: biên màn hình, paddle, và các viên gạch.
- **Điều khiển âm thanh:** Kích hoạt Buzzer thông qua tín hiệu PWM (chân PF8) để phát ra các âm báo đặc trưng (tone) khác nhau cho từng sự kiện: Chạm biên, phá gạch, Game Over...

2.2 Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng các tiêu chuẩn về hiệu năng và trải nghiệm người dùng:

- **Tốc độ phản hồi (Real-time Performance):**
 - Độ trễ giữa thao tác xoay biến trở và chuyển động của paddle trên màn hình phải cực thấp (gần như tức thời) để người chơi có thể phản xạ kịp với tốc độ bóng.

- Tốc độ khung hình (FPS - Frames Per Second) phải đủ cao (tối thiểu 24 – 30 FPS) để chuyển động của bóng mượt mà, không bị giật cục.
- **Độ chính xác:**
 - Thuật toán vật lý (hướng nảy của bóng) phải hợp lý, không để bóng bị kẹt trong tường hoặc paddle.
 - Giá trị ADC từ biến trở phải ổn định, hạn chế nhiễu để paddle không bị rung lắc khi người chơi giữ nguyên tay.
- **Giao diện người dùng (User Interface):**
 - Bố cục màn hình rõ ràng, dễ quan sát. Font chữ hiển thị điểm số và menu phải dễ đọc.
 - Tránh hiện tượng nhấp nháy (flicker) khi cập nhật lại màn hình đồ họa.
- **Tính ổn định:** Hệ thống hoạt động liên tục trong thời gian dài mà không bị treo (crash) hoặc tràn bộ nhớ (stack overflow), đặc biệt là khi xử lý nhiều đối tượng gạch cùng lúc.

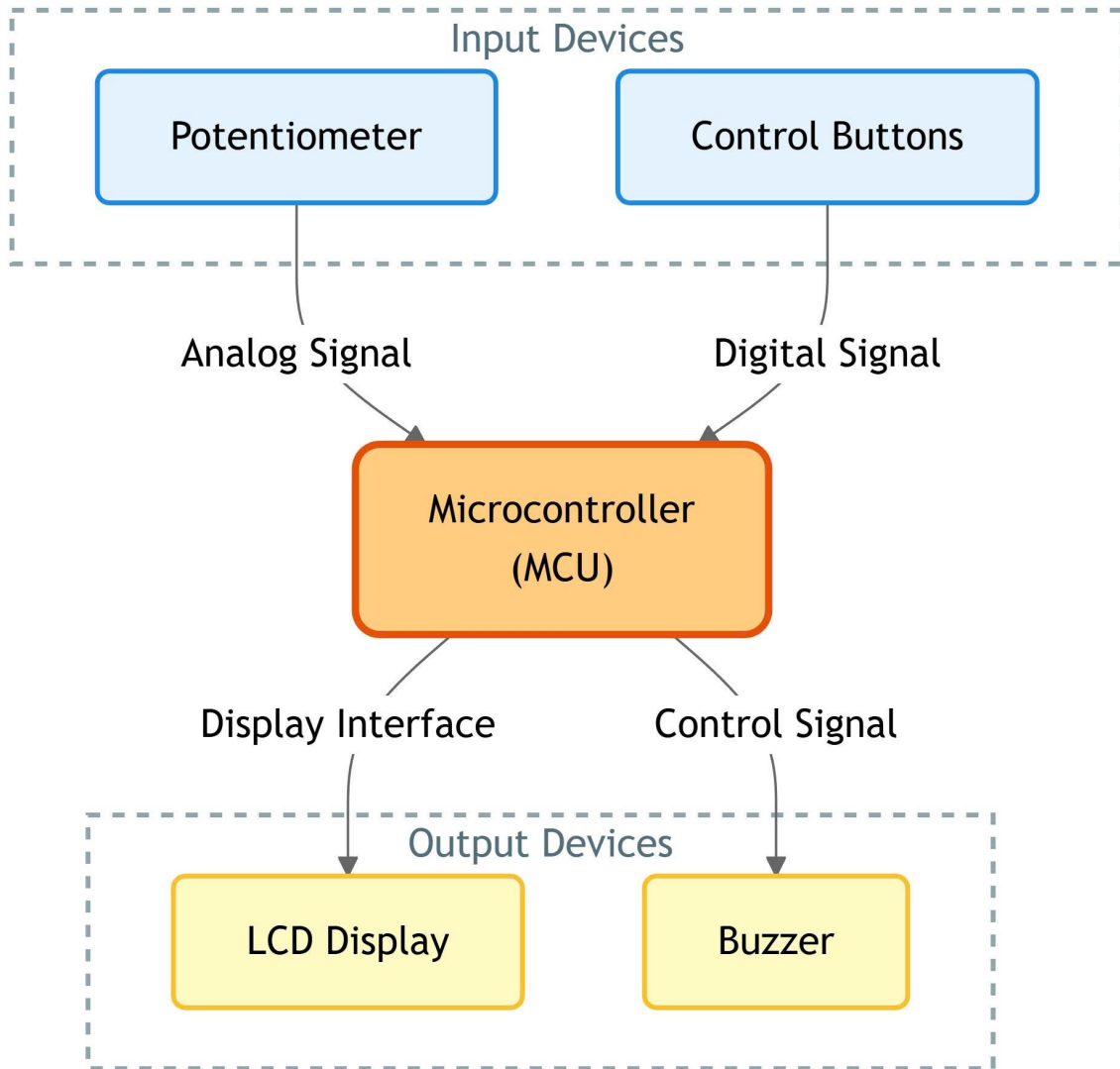
2.3 Môi trường hoạt động

Hệ thống được thiết kế để hoạt động trong môi trường thực nghiệm với các thông số kỹ thuật sau:

- **Phần cứng vận hành:**
 - Kit phát triển: BKIT-Arm4 (dựa trên vi điều khiển STM32).
 - Vi xử lý trung tâm: ARM Cortex-M (32-bit).
- **Nguồn điện:** Sử dụng nguồn 5V DC cấp từ cổng USB kết nối với máy tính hoặc Adapter rời. Yêu cầu nguồn điện ổn định để đảm bảo độ chính xác cho bộ ADC và độ sáng màn hình LCD.
- **Kết nối ngoại vi:** Hệ thống hoạt động độc lập (Standalone), các kết nối là kết nối nội bộ trên mạch (On-board) thông qua các chuẩn giao tiếp công nghiệp: I2C, SPI/FSMC, GPIO, ADC, PWM. Không yêu cầu kết nối mạng (WiFi/Ethernet).
- **Điều kiện môi trường:**
 - Nhiệt độ và độ ẩm phòng thí nghiệm tiêu chuẩn.
 - Ánh sáng môi trường vừa đủ để quan sát màn hình LCD (do màn hình có đèn nền).

3 Thiết kế hệ thống

3.1 Sơ đồ khối tổng thể của hệ thống



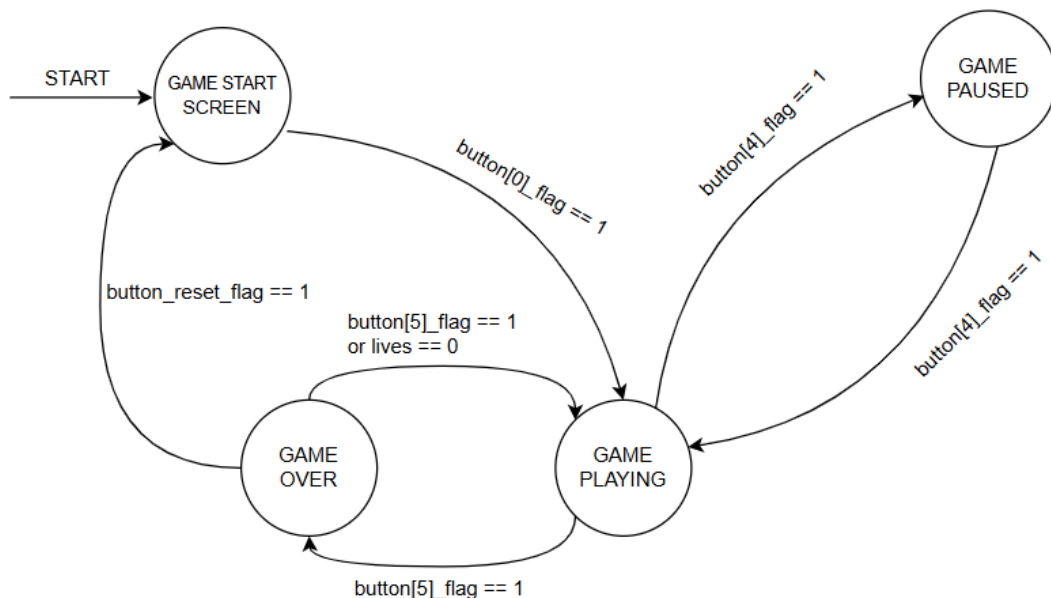
Hình 1: Sơ đồ khối tổng thể của hệ thống

Hệ thống được thiết kế theo mô hình tập trung với Vi điều khiển (MCU) làm trung tâm điều phối. Các thành phần phần cứng được chia thành 4 khối chức năng chính như sau:

- Khối Xử lý Trung tâm (Microcontroller - MCU), đây là khối quan trọng nhất, sử dụng vi điều khiển STM32 trên kit BKIT-Arm4. MCU chịu trách nhiệm:
 - Thu thập dữ liệu từ biến trở và nút nhấn.
 - Xử lý logic trò chơi: tính toán vật lý bóng, phát hiện va chạm, cập nhật trạng thái game.
 - Điều khiển các thiết bị ngoại vi đầu ra và quản lý bộ nhớ.

- Khối Đầu vào (Input Devices), khối này cung cấp phương thức tương tác giữa người chơi và hệ thống:
 - **Biến trở (Potentiometer):** Gửi tín hiệu *Analog Signal* về chân ADC của MCU. Giá trị điện áp thay đổi khi người chơi xoay biến trở sẽ được chuyển đổi thành tọa độ vị trí của thanh trượt (Paddle).
 - **Nút nhấn (Buttons):** Gửi tín hiệu *Digital Signal* (Logic 0/1) về các chân GPIO để điều khiển trạng thái hệ thống.
- Khối Đầu ra (Output Devices), khối này phản hồi thông tin từ hệ thống đến người chơi:
 - **Màn hình LCD:** Kết nối qua giao diện hiển thị (*Display Interface*), chịu trách nhiệm hiển thị đồ họa trò chơi và thông số điểm số.
 - **Còi chirp (Buzzer):** Nhận tín hiệu điều khiển (*Control Signal*) dạng xung PWM để tạo hiệu ứng âm thanh cho các sự kiện trong game.

3.2 Thiết kế luồng thực thi của hệ thống dựa vào Máy trạng thái (FSM)



Hình 2: Sơ đồ máy trạng thái của hệ thống

Phần mềm điều khiển trò chơi được thiết kế dựa trên mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM). Mô hình này đảm bảo chương trình hoạt động ổn định và quản lý chặt chẽ các giai đoạn của trò chơi. Hệ thống bao gồm 4 trạng thái chính như được mô tả trong sơ đồ chuyển trạng thái.

- Các trạng thái hoạt động:

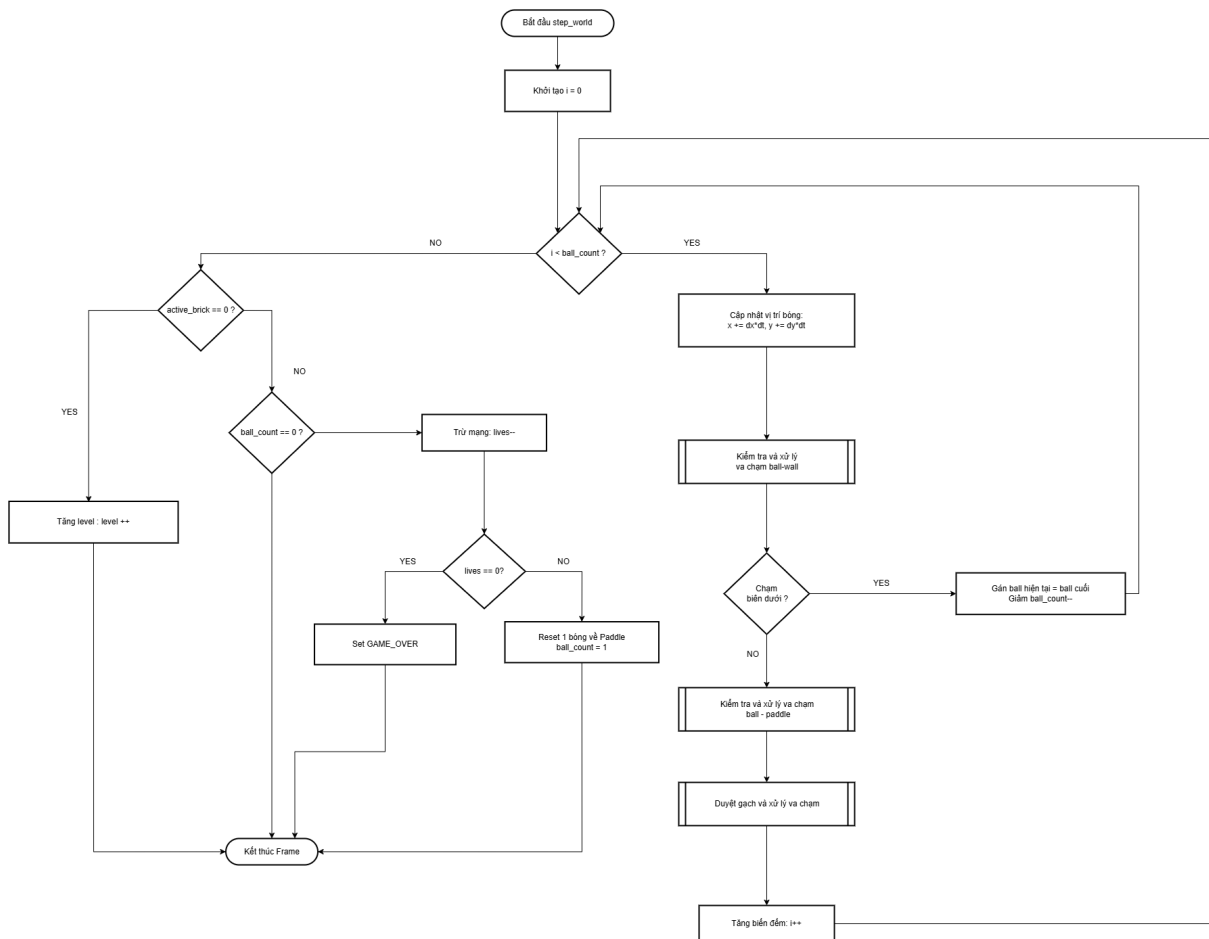
1. **GAME START SCREEN (Màn hình chờ):** Đây là trạng thái khởi tạo khi cấp nguồn. Hệ thống hiển thị giao diện chào mừng và chờ người dùng bắt đầu.
2. **GAME PLAYING (Đang chơi):** Trạng thái hoạt động chính. Vi điều khiển thực hiện vòng lặp vô hạn để cập nhật vị trí bóng, đọc biến trở điều khiển paddle, kiểm tra va chạm và vẽ lại màn hình hiển thị.
3. **GAME PAUSED (Tạm dừng):** Trạng thái ngắt tạm thời. Mọi tính toán vật lý dừng lại, vị trí các đối tượng được giữ nguyên.
4. **GAME OVER (Kết thúc):** Trạng thái khi người chơi mất hết mạng (lives). Hệ thống hiển thị điểm số cuối cùng và chờ thao tác tiếp theo từ người dùng.

- Logic chuyển đổi trạng thái (State Transitions), việc chuyển đổi giữa các trạng thái được kích hoạt bởi các cờ hiệu (flags) từ các nút nhấn (Input Buttons):

- Bắt đầu game: Từ **GAME START SCREEN** $\xrightarrow{\text{button}[0]_{\text{flag}}==1}$ **GAME PLAYING**.
- Tạm dừng / Tiếp tục:
 - * Từ **GAME PLAYING** $\xrightarrow{\text{button}[4]_{\text{flag}}==1}$ **GAME PAUSED**.
 - * Từ **GAME PAUSED** $\xrightarrow{\text{button}[4]_{\text{flag}}==1}$ quay lại **GAME PLAYING**.
- Xử lý thua cuộc (Game Over): Từ **GAME PLAYING** $\xrightarrow{\text{button}[5]_{\text{flag}}==1}$ **GAME OVER**. Bên cạnh đó có thể được chuyển tự động khi biến **lives** về 0.
- Tác vụ trong game: Tại **GAME PLAYING**, nếu ta xoay biến trở, hệ thống thực hiện hành động bổ trợ (ví dụ: tính năng bắn bóng) và duy trì trạng thái hiện tại (Self-loop).
- Điều hướng sau khi kết thúc:
 - * Chơi lại ngay (Replay): Từ **GAME OVER** $\xrightarrow{\text{button}[5]_{\text{flag}}==1}$ **GAME PLAYING**.
 - * Reset về menu: Từ **GAME OVER** $\xrightarrow{\text{button_reset_flag}==1}$ **GAME START SCREEN**.

3.3 Thiết kế Flowchart cho xử lý logic game

3.3.1 Flowchart Main Loop (step_world)

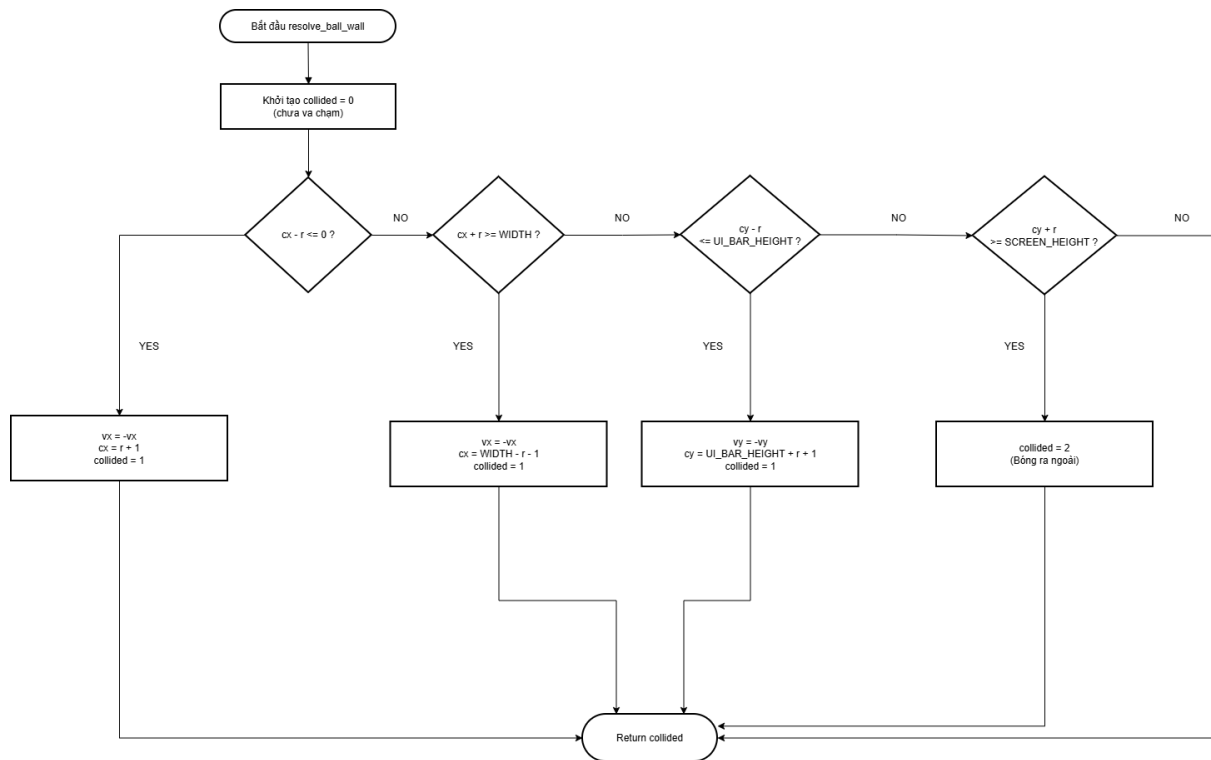


Hình 3: FFlowchart xử lý logic cho mỗi khung hình

Lưu đồ mô tả quy trình xử lý trong một khung hình, bao gồm: vòng lặp cập nhật vị trí các quả bóng, gọi các hàm kiểm tra va chạm, xử lý logic khi bóng chạm biên dưới, và kiểm tra điều kiện thắng (hết gạch) hoặc thua (hết mạng).

Tốc độ quét và xử lý khung hình cần phải đủ nhanh thì mới cho cảm giác xử lý mượt và chính xác. Do vậy nhóm chọn tốc độ quét khung hình là 20ms, tương đương 50 FPS, đủ để tạo cảm giác mượt mà khi chơi.

3.4 Flowchart Wall Collision (resolve_ball_wall)

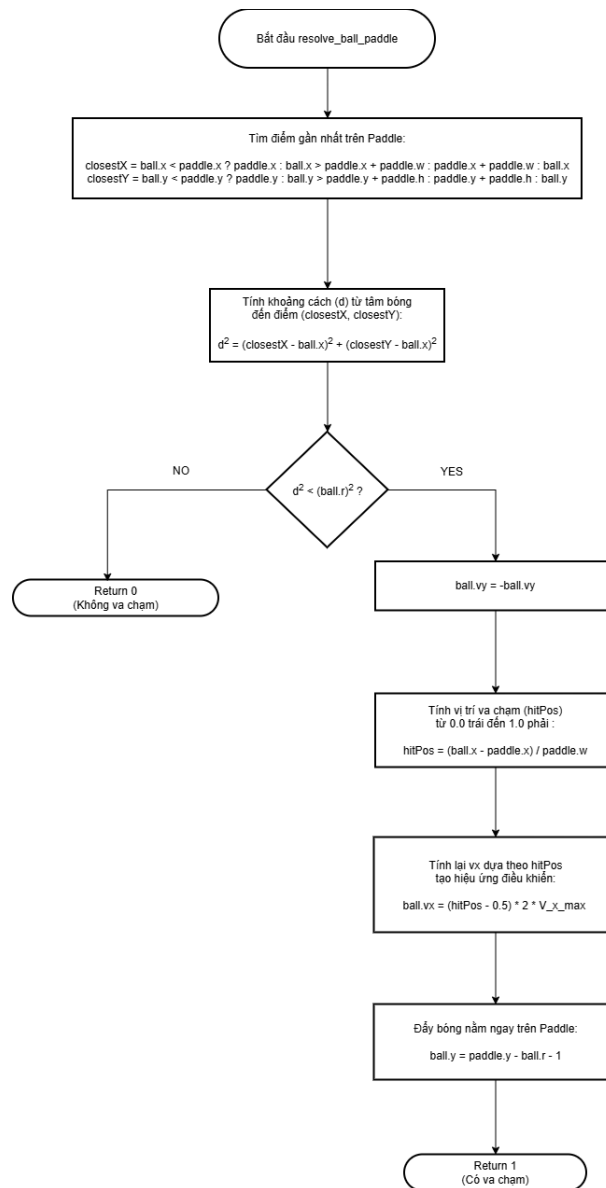


Hình 4: Flowchart xử lý va chạm với biên màn hình

Thuật toán kiểm tra tọa độ của bóng so với 4 cạnh màn hình. Nếu chạm biên trái, phải hoặc trên, vận tốc của bóng sẽ được đảo chiều để tạo hiệu ứng nảy. Nếu chạm biên dưới, bóng được xác định là rơi ra ngoài (Out of bounds). Các giá trị trả về của `collided` là 0, 1, 2 lần lượt tương đương với không va chạm; va chạm cạnh 2 cạnh bên hoặc trên; và chạm biên dưới.

Thuật toán tiếp xử lý sau va chạm để bóng không bị kẹt tại biên bằng cách đặt tọa độ bóng lại sát biên đồng thời xác định hướng vận tốc của bóng tùy theo loại va chạm với tường.

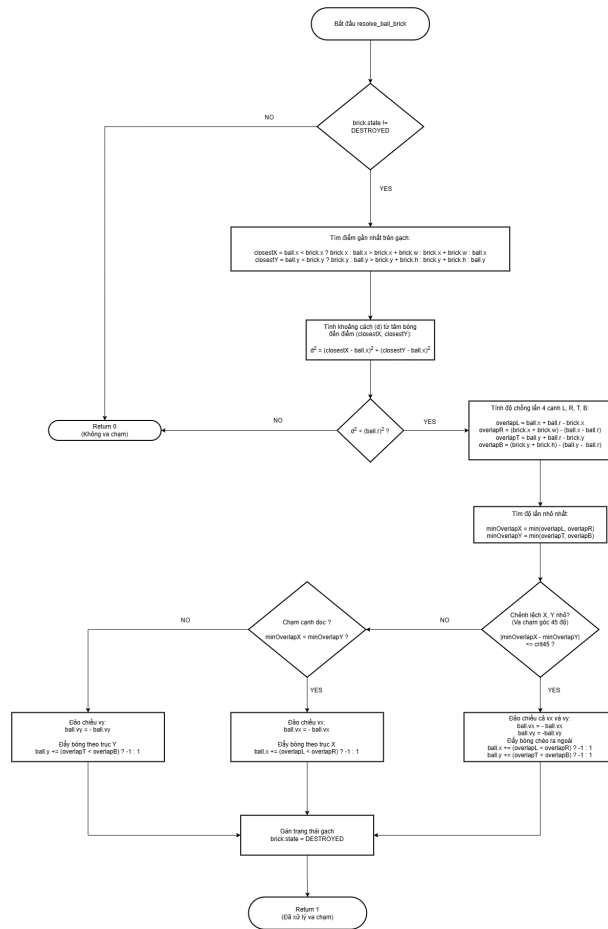
3.5 Flowchart Paddle Collision (resolve_ball_paddle)



Hình 5: Flowchart xử lý va chạm giữa bóng và thanh đỡ (resolve_ball_paddle) sử dụng kỹ thuật AABB

Lưu đồ thể hiện chi tiết thuật toán AABB (Axis-Aligned Bounding Box) để phát hiện va chạm giữa hình tròn (bóng) và hình chữ nhật (thanh đỡ). Sau khi xác định va chạm, thuật toán tính toán vị trí tiếp xúc (Hit Position) để điều chỉnh góc phản xạ, tạo cảm giác điều khiển vật lý chân thực cho người chơi, cho phép người chơi dùng paddle để điều khiển có mục đích hơn. Tương tự như wall-collision, thuật toán cũng đặt lại vị trí bóng nằm ngay trên paddle để không bị kẹt.

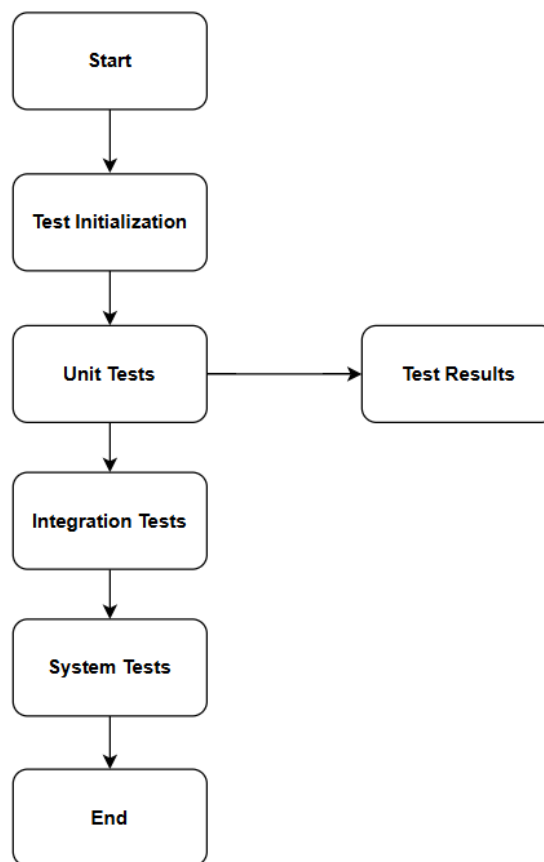
3.6 Flowchart Brick Collision (resolve_ball_brick)



Hình 6: Flowchart xử lý va chạm và phản xạ với gạch (resolve_ball_brick)

Quy trình xử lý va chạm với các viên gạch. Trước tiên dùng thuật toán AABB để xác định có va chạm giữa bóng và gạch không. Nếu có va chạm, thuật toán tính toán độ chồng lấn (Overlap) giữa bóng đối với 4 cạnh của gạch để xác định chính xác hướng va chạm (ngang, dọc hoặc góc 45 độ) với **crit45** là ngưỡng xác định va chạm góc được tính toán từ các công thức hình học và xấp xỉ $\cos 45^\circ \cdot r$, từ đó quyết định hướng phản xạ của bóng và cập nhật trạng thái hủy của viên gạch. Tương tự 2 thuật toán va chạm trên, vị trí bóng cũng được cập nhật lại để không bị kẹt trong gạch.

3.7 Thiết kế Testcase cho hệ thống



Hình 7: Quy trình kiểm thử hệ thống

Quy trình kiểm thử hệ thống được thực hiện theo mô hình phân cấp từ thấp đến cao như minh họa trong sơ đồ *Flow test*. Các bước cụ thể bao gồm:

- **Test Initialization (Khởi tạo):** Thiết lập cấu hình phần cứng BKIT-Arm4, nạp firmware và chuẩn bị môi trường gỡ lỗi (debug).
- **Unit Tests (Kiểm thử đơn vị):** Tiến hành kiểm tra chức năng độc lập của từng ngoại vi (Peripheral).
 - Kiểm tra hiển thị màu sắc, hình khối trên LCD.
 - Kiểm tra độ chính xác giá trị đọc về từ ADC và trạng thái nút nhấn GPIO.
 - *Kết quả (Test Results):* Đánh giá Pass/Fail cho từng module.
- **Integration Tests (Kiểm thử tích hợp):** Kiểm tra sự phối hợp giữa các khối Input và Output. Như là: Đảm bảo khi nhấn nút "Start", màn hình chuyển từ Menu sang giao diện chơi game, còi chip phát âm thanh...

- **System Tests (Kiểm thử hệ thống):** Thực hiện chơi thử nghiệm (User Acceptance Testing) để đánh giá độ mượt của chuyển động bóng, độ nhạy của paddle và tính ổn định của logic game trong thời gian dài.

Kịch bản và Kết quả kiểm thử :

Để đảm bảo hệ thống hoạt động ổn định và đáp ứng đúng yêu cầu thiết kế, nhóm thực hiện đã tiến hành kiểm thử theo hai cấp độ: Kiểm thử đơn vị (Unit Test) và Kiểm thử tích hợp (Integration Test).

- Kiểm thử đơn vị (Unit Test) Mục đích: Kiểm tra tính đúng đắn của các hàm xử lý logic cốt lõi và các module phần cứng riêng biệt.

ID	Mục tiêu	Input / Điều kiện	Kết quả kỳ vọng
TC01	Test bóng va chạm tường	$Ball.x = 0$ hoặc $Ball.x = Max$, $v_x < 0$	Vận tốc v_x đảo chiều, bóng nảy ngược lại.
TC02	Test phá gạch	Bóng chạm tọa độ $Brick[i][j]$	$Brick[i][j]$ bị hủy (biến mất), biến Score tăng lên.
TC03	Test power-up	Một viên gạch chứa power-up bị phá vỡ	Tự động kích hoạt phần thưởng đó.
TC04	Test di chuyển	Xoay biến trở	Thanh trượt (Paddle) dịch chuyển sang phải tương ứng.

Bảng 2: Bảng kịch bản Unit Test

- Kiểm thử tích hợp (Integration Test) Mục đích: Kiểm tra sự tương tác giữa các thành phần (Module) khác nhau như Input, Logic, Graphics, Sound và Memory.

ID	Mục tiêu	Mô tả kiểm tra	Kỳ vọng
1. Input ↔ Paddle			
IT01	Đọc biến trở di chuyển paddle	Xoay biến trở trong khoảng 0–3.3V, quan sát LCD	Paddle di chuyển mượt mà, không rung lắc, không vượt quá biên màn hình.
IT02	Độ trễ input	Xoay nhanh biến trở	Paddle phản hồi ngay lập tức ($\text{độ trễ} \leq 100\text{ms}$).
2. Game Logic ↔ LCD Graphics			
IT03	Hiển thị khởi tạo	Reset game	Màn hình "Menu" hiển thị đúng layout, màu sắc.
IT04	Vẽ và cập nhật bóng/-paddle	Trong trạng thái "Playing"	Bóng và paddle cập nhật theo vị trí thực, hình ảnh không bị nhấp nháy (flicker).

ID	Mục tiêu	Mô tả kiểm tra	Kỳ vọng
IT05	Collision → Bricks	Khi bóng chạm gạch	Gạch biến mất trên màn hình, điểm số cập nhật tăng, âm thanh phát ra.
IT06	Collision → Floor	Bóng chạm đáy màn hình	Mất 1 mạng (Lives giảm), game tiếp tục hoặc Game Over nếu hết mạng.
3. Sound ↔ Game Events			
IT07	Âm thanh va chạm	Khi bóng đập vào gạch hoặc paddle	Âm thanh PWM được phát đúng tần số/cao độ thiết lập.
IT8	Âm thanh Game Over	Khi số mạng về 0	Âm thanh đặc trưng được phát rõ ràng, không bị méo tiếng.
4. Power-ups ↔ Logic			
IT9	Nhận vật phẩm	Khi gạch đặc biệt (nếu có) bị phá	Kích hoạt phần thưởng nhảy lập tức.
IT10	Áp dụng hiệu ứng	Khi paddle chạm vật phẩm	Hiệu ứng kích hoạt đúng (VD: tăng tốc bóng, nhân đôi bóng).
IT11	Hết hiệu ứng	Sau thời gian hiệu lực	Tất cả trạng thái trở về bình thường.
6. System Integration			
IT12	FPS / Lag check	Chạy full game với 20+ viên gạch	Game chạy ổn định 60 FPS, không có hiện tượng giật lag rõ rệt.
IT13	Reset game	Nhấn nút Reset trên board	Toàn hệ thống khởi tạo lại đầy đủ (RAM cleared).
IT14	Power cycle	Tắt/Mở nguồn điện	Game load lại thành công.

Bảng 3: Kế hoạch và kết quả kiểm thử tích hợp

Bạn em có hiện thực các trường hợp kiểm thử Unit Test (Logic Vật lý). Các kiểm thử này tập trung vào việc xác minh tính đúng đắn của bộ xử lý vật lý (Physics Engine) và phát hiện va chạm trong game. Truy cập đường link sau để đến với mã nguồn Unit Test: [link](#)

3.8 Thiết kế giao diện cho hệ thống

3.8.1 Tổng quan về Bố cục và Các Thành phần Chính

Giao diện trò chơi được thiết kế tối giản, tập trung vào trải nghiệm chơi game. Các thành phần đồ họa chính được hiển thị trên màn hình bao gồm:

- **Lưới Gạch:** Được bố trí ở nửa trên màn hình, tạo thành chướng ngại vật chính của trò chơi.
- **Thanh trượt:** Một thanh ngang nằm ở phía dưới màn hình và được điều khiển bởi người chơi.
- **Bóng:** Một vật thể hình tròn di chuyển và tương tác với các thành phần khác.
- **Khu vực Hiển thị Thông tin (HUD):** Hiển thị điểm số và mạng sống, thường đặt tại góc trên hoặc dưới màn hình.

3.8.2 Mô tả các Màn hình Giao diện

Trò chơi bao gồm 5 màn hình chính, mỗi màn hình có giao diện trực quan để thể hiện trạng thái của trò chơi.

3.8.2.1 Màn hình Giới thiệu (Splash Screen)

Mục đích: Giới thiệu tên trò chơi và đơn vị chủ quản (Trường Đại học).

Mô tả:

- **Nền:** Toàn bộ màn hình sử dụng tông màu xanh ngọc (mint green) làm chủ đạo.
- **Logo:** Phía trên cùng hiển thị logo của trường Đại học Bách Khoa ĐHQG-HCM (BK TP.HCM) nằm trong khung nền trắng.
- **Tên trò chơi:** Ở chính giữa là dòng chữ "**BRICK BREAKER**" được viết in hoa, phông chữ lớn, nét đậm với viền đen tạo hiệu ứng nổi (3D).
- **Hình minh họa:** Phía dưới cùng là hình ảnh đồ họa đơn giản của một bức tường gạch màu cam đỏ với các đường viền đen đậm, tượng trưng cho lối chơi phá gạch.



Hình 8: Màn hình giới thiệu

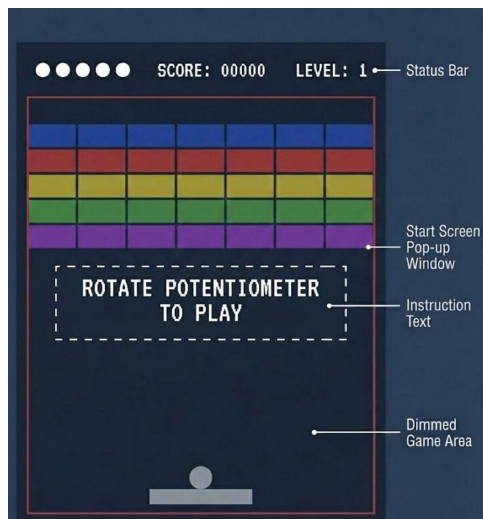
3.8.2.2 Màn hình Bắt đầu

Mục đích: Chào mừng người chơi và cung cấp hướng dẫn điều khiển cơ bản.

Mô tả:

- Nền là màn hình chơi game ban đầu với đầy đủ gạch, thanh trượt và bóng ở vị trí xuất phát.
- Ở giữa màn hình là hộp thoại hướng dẫn với viền nét đứt để tạo điểm nhấn.
- Bên trong hộp thoại hiển thị dòng chữ:

ROTATE POTENTIOMETER
TO PLAY



Hình 9: Hình ảnh Hướng Dẫn chơi game

3.8.2.3 Màn hình Chơi game

Mục đích: Đây là giao diện chính nơi diễn ra mọi hành động.

Mô tả:

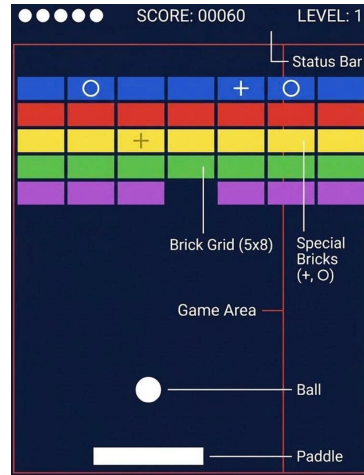
- **Lưới gạch:** Các viên gạch hình chữ nhật được sắp xếp dạng hàng và cột.
- **Thanh trượt:** Di chuyển mượt mà theo chiều ngang ở cuối màn hình.
- **Bóng:** Di chuyển liên tục tạo cảm giác hành động.
- **Điểm số:** Hiển thị điểm hiện tại của người chơi.
- **Mạng sống:** Số mạng còn lại, hiển thị bằng số hoặc biểu tượng (ví dụ: hình tròn).

Để trò chơi hấp dẫn hơn, một số viên gạch có thiết kế đặc biệt để phân biệt với gạch thông thường:

- **Gạch thưởng Bóng:** Có một vòng tròn màu trắng ở giữa.

- **Gạch thưởng “Plus”**: Có dấu cộng (+) màu đen ở chính giữa.

Những thiết kế này giúp người chơi nhận biết các phần thưởng tiềm năng và tạo động lực nhắm vào chúng.



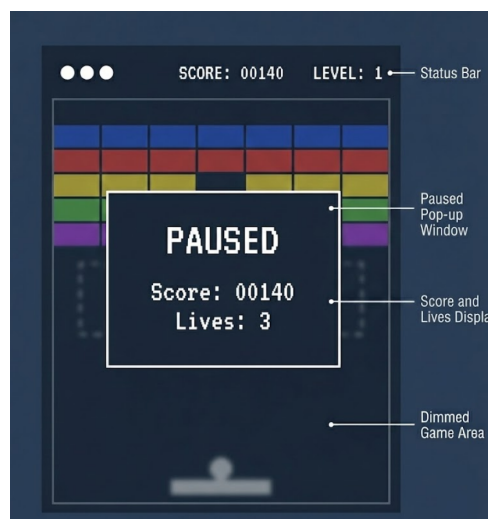
Hình 10: Hình ảnh chơi game

3.8.2.4 Màn hình Tạm dừng

Mục đích: Thông báo cho người chơi biết game đang tạm dừng.

Mô tả:

- Một lớp phủ (overlay) xuất hiện trên Màn hình Chơi game.
- Hình ảnh bên dưới có thể được làm mờ hoặc giữ nguyên.
- Ở giữa màn hình hiển thị dòng chữ lớn: **PAUSED**.
- Điểm số và mạng sống vẫn có thể hiển thị.



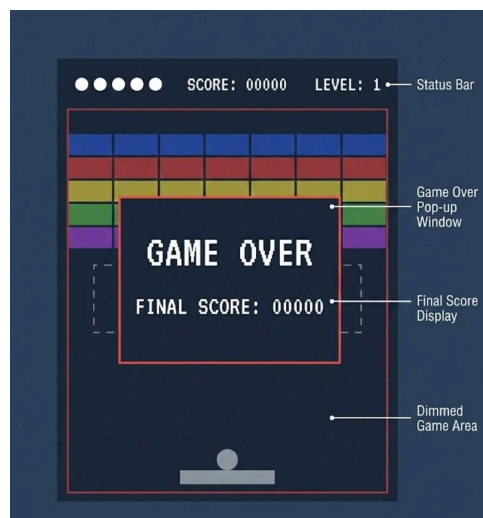
Hình 11: Hình Ảnh Pause Game

3.8.2.5 Màn hình Kết thúc (Game Over)

Mục đích: Thông báo trò chơi đã kết thúc và hiển thị thành tích cuối cùng.

Mô tả:

- Màn hình được xóa sạch.
- Một dòng chữ lớn ở trung tâm: **GAME OVER**.
- Ngay bên dưới hiển thị điểm số cuối cùng mà người chơi đạt được.



Hình 12: Hình Ảnh Game Over

4 Triển khai

4.1 Mô tả chi tiết phần cứng sử dụng

Hệ thống trò chơi Brick Breaker được triển khai trên Kit phát triển **BKIT-Arm4**. Đây là nền tảng phần cứng tiêu chuẩn được sử dụng trong các bài thí nghiệm hệ thống nhúng, tích hợp đầy đủ các ngoại vi cần thiết cho dự án.

- Vi điều khiển trung tâm (MCU)
 - **Dòng chip:** STM32F407ZGT6 (ARM Cortex-M4 32-bit).
 - **Tần số hoạt động:** 168 MHz.
 - **Bộ nhớ:** 1MB Flash và 192KB SRAM.
 - **Vai trò:** Chịu trách nhiệm xử lý toàn bộ logic game, tính toán vật lý va chạm và điều phối các ngoại vi. Đơn vị FPU tích hợp giúp tăng tốc độ tính toán số thực cho các vector chuyển động của bóng.
- Module hiển thị (Display)
 - **Thiết bị:** Màn hình TFT LCD 2.4 inch, Driver ILI9341.
 - **Độ phân giải:** 240×320 pixels, chuẩn màu 16-bit RGB565.
 - **Giao tiếp:** FSMC (Flexible Static Memory Controller) 16-bit song song.
 - **Lý do lựa chọn:** Giao tiếp FSMC cung cấp băng thông lớn, cho phép tốc độ làm tươi màn hình cao (High Refresh Rate), đảm bảo chuyển động của bóng mượt mà, không bị hiện tượng bóng mờ.
- Các ngoại vi khác
 - **Input - Biến trở (Potentiometer):** Kết nối với bộ chuyển đổi ADC 12-bit. Dùng để điều khiển vị trí thanh trượt (Paddle) dựa trên sự thay đổi điện áp tương tự.
 - **Output - Còi chirp (Buzzer):** Điều khiển bằng tín hiệu PWM (Pulse Width Modulation). Thay đổi tần số PWM để tạo ra các hiệu ứng âm thanh khác nhau cho sự kiện va chạm và kết thúc game.

4.2 Công cụ phát triển

Quá trình phát triển phần mềm dựa trên hệ sinh thái công cụ của STMicroelectronics:

- STM32CubeIDE, là môi trường phát triển tích hợp (Integrated Development Environment) chính:
 - **Soạn thảo & Biên dịch:** Hỗ trợ ngôn ngữ C/C++, quản lý thư viện HAL (Hardware Abstraction Layer).
 - **STM32CubeMX:** Công cụ cấu hình đồ họa tích hợp, giúp thiết lập Clock, GPIO, ADC, Timer và sinh mã khởi tạo tự động.

- **Debugging:** Hỗ trợ gỡ lỗi thời gian thực (Real-time debugging), cho phép theo dõi giá trị biến (Score, Lives, Ball Coordinates) và thanh ghi ngay khi game đang chạy.
- STM32CubeProgrammer, công cụ quản lý bộ nhớ và nạp chương trình:
 - Kết nối với mạch nạp ST-Link V2 trên board.
 - Nạp file thực thi (.hex, .elf) vào bộ nhớ Flash của vi điều khiển.
 - Kiểm tra tính toàn vẹn của dữ liệu sau khi nạp.

4.3 Mã nguồn hiện thực chương trình

Để xem chi tiết cách hiện thực, truy cập đường link sau: [link](#)

5 Kết quả và đánh giá

5.1 Kết quả Demo

Hệ thống đã được hoàn thiện phần cứng và phần mềm, vận hành ổn định trên Kit phát triển BKIT-Arm4. Dưới đây là minh chứng hoạt động thực tế của sản phẩm:

- **Nội dung demo:** Video minh họa quy trình chơi game hoàn chỉnh, bao gồm các thao tác: Vào menu chính, chơi game (điều khiển paddle, phá gạch), tạm dừng, xử lý khi thua cuộc (Game Over), qua level tiếp theo khi phá hết gạch.
- **Link Demo:** [link](#)

5.2 Đánh giá hiệu năng

Dựa trên quá trình kiểm thử hệ thống (System Testing), nhóm thực hiện đưa ra các đánh giá định lượng và định tính sau:

- Độ mượt mà và Đồ họa
 - Tốc độ khung hình (FPS) duy trì ổn định ở mức cao (ước tính > 30 FPS) nhờ tối ưu hóa giao tiếp FSMC và thuật toán vẽ chỉ cập nhật vùng thay đổi (Dirty Rectangle).
 - Không xuất hiện hiện tượng giật (lag) hay xé hình khi bóng di chuyển ở tốc độ cao.
- Độ trễ và Điều khiển
 - Hệ thống phản hồi tức thời với thao tác xoay biến trở. Độ trễ (Latency) từ Input đến Output thấp dưới ngưỡng nhận biết của người dùng, tạo cảm giác điều khiển mượt mà.
 - Các nút nhấn chức năng (Pause, Reset) hoạt động chính xác nhờ giải thuật khử rung (De-bouncing) tốt.
- Độ ổn định
 - Hệ thống hoạt động bền bỉ trong các bài test chạy dài (Long-run test > 30 phút) mà không gặp lỗi treo máy hay rò rỉ bộ nhớ (Memory Leak).
 - Dữ liệu điểm cao trong EEPROM được bảo toàn chính xác qua nhiều lần tắt/mở nguồn.

5.3 So sánh với mục tiêu ban đầu

Bảng dưới đây đối chiếu kết quả thực tế đạt được so với các mục tiêu đề ra ở giai đoạn khởi động dự án:

STT	Mục tiêu đề ra	Hoàn thành	Đánh giá
1	Xây dựng Gameplay cơ bản	100%	Logic vật lý và va chạm hoạt động đúng.
2	Hiển thị đồ họa trên LCD (ILI9341)	100%	Hiển thị sắc nét, không nhấp nháy.
3	Điều khiển Analog qua Biến trở	100%	Ánh xạ giá trị ADC chính xác.
4	Hiệu ứng âm thanh (Buzzer)	100%	Âm thanh đồng bộ với hình ảnh.
5	Tính năng mở rộng (Power-ups)	100%	Các vật phẩm hỗ trợ xuất hiện và có hiệu lực đúng thiết kế.

Bảng 4: Bảng so sánh kết quả với mục tiêu ban đầu

6 Kết luận và hướng phát triển

6.1 Tổng kết điều đạt được

Sau quá trình nghiên cứu và triển khai, đề tài "Xây dựng trò chơi Brick Breaker trên Kit BKIT-Arm4" đã hoàn thành các mục tiêu đề ra ban đầu. Nhóm thực hiện đã đạt được những kết quả cụ thể sau:

- **Về mặt kỹ thuật:** Đã làm chủ được kiến trúc vi điều khiển ARM Cortex-M4 (STM32F407) và các chuẩn giao tiếp ngoại vi quan trọng như GPIO, ADC, PWM, FSMC và I2C.
- **Về mặt sản phẩm:** Xây dựng thành công một hệ thống nhúng hoàn chỉnh, kết hợp chặt chẽ giữa phần cứng và phần mềm. Trò chơi vận hành mượt mà, logic và chạm chính xác và giao diện người dùng trực quan.
- **Về mặt kỹ năng:** Nâng cao tư duy lập trình hệ thống nhúng thời gian thực (Real-time), kỹ năng gỡ lỗi (Debugging) phần cứng và quản lý tài nguyên bộ nhớ hạn chế.

6.2 Các giới hạn của hệ thống

Mặc dù hoạt động ổn định, hệ thống vẫn tồn tại một số hạn chế khách quan:

- **Hạn chế về âm thanh:** Do sử dụng Buzzer thụ động điều khiển bằng xung PWM, hệ thống chỉ có thể phát ra các âm báo đơn sắc (Beep tones). Trải nghiệm âm thanh chưa thực sự sống động (thiếu nhạc nền, giọng nói).
- **Hạn chế về kết nối:** Hệ thống hoạt động ở chế độ độc lập (Standalone). Việc thiếu các module giao tiếp không dây (WiFi/Bluetooth) khiến game không có tính năng bảng xếp hạng trực tuyến (Online Leaderboard) hay chế độ nhiều người chơi (Multiplayer).
- **Hạn chế về đồ họa:** Việc xử lý đồ họa hoàn toàn dựa trên CPU (Software Rendering) giới hạn khả năng hiển thị các hiệu ứng phức tạp (như hạt cháy nổ, bóng đổ) mà không làm giảm tốc độ khung hình (FPS).

6.3 Đề xuất cải tiến

Để nâng cao chất lượng và khả năng ứng dụng thực tế của đề tài, nhóm đề xuất các phương hướng phát triển trong tương lai:

- **Tích hợp IoT:** Bổ sung module WiFi (như ESP8266/ESP32) kết nối qua UART để gửi dữ liệu điểm cao lên Server/Cloud, cho phép người chơi so sánh thành tích toàn cầu.
- **Ghi lại điểm cao nhất của người chơi:** Việc ghi điểm cao vào bộ nhớ EEPROM mô phỏng quy trình lưu trữ dữ liệu cấu hình (Configuration Data) hoặc nhật ký hoạt động (Data Logging) trong các thiết bị IoT (Internet of Things), đảm bảo dữ liệu không bị mất khi ngắt nguồn.
- **Nâng cấp phương thức điều khiển:** Sử dụng cảm biến gia tốc (Accelerometer - ví dụ MPU6050) để điều khiển thanh trượt bằng thao tác nghiêng mạch, tạo trải nghiệm chơi mới lạ.

- **Tối ưu hóa hiệu năng:** Áp dụng cơ chế DMA (Direct Memory Access) cho việc truyền dữ liệu xuống màn hình LCD và sử dụng I2S cho module âm thanh rời (Audio Codec) để nâng cao chất lượng nghe nhìn.
- **Mở rộng Gameplay:** Thiết kế trình biên tập màn chơi (Level Editor) cho phép lưu cấu trúc các màn chơi phức tạp vào thẻ nhớ SD Card thay vì mã hóa cứng trong chương trình.

7 Tài liệu tham khảo

Tài liệu tham khảo

- [1] ABC Solutions, “Giới thiệu Kit thí nghiệm BKIT-ARM4,”. Truy cập: <https://abcsolutions.com.vn/index.php/kit-thi-nghiem-bkit-arm4/>.
- [2] ABC Solutions, “Tài liệu hướng dẫn nhanh: KitThiNghiem_STM32_ARM4_QuickStartGuide,”. Truy cập: https://abcsolutions.com.vn/wp-content/uploads/2023/11/KitThiNghiem_STM32_ARM4_QuickStartGuide_20231027_1027.pdf.
- [3] ABC Solutions, “Kho tài liệu kỹ thuật, Driver và Mã nguồn mẫu cho BKIT-Arm4,”. Truy cập: <https://drive.google.com/drive/folders/1-nrxDofZvNRVR-uWFto3qvDx2LCEXZcE>.